

События

Понятие события

События - это функции, которые могут быть привязаны к элементам HTML страниц.

Код событий выполнится только после того, как произойдет их **активирующее действие**. Разные типы событий имеют разные активирующие действия.

Примеры активирующих действий JavaScript:

- Щелчок мыши (*событие onclick*);
- Нажатие клавиши (*onkeypress*);
- Отправление формы (*onsubmit*);
- Наведение курсора мыши на элемент (*onmouseover*) или выведение курсора мыши за пределы границ элемента (*onmouseout*);
- Полная загрузка страницы или картинки (*onload*);
- Изменение содержимого элемента, например содержимого текстового поля формы (*onchange*).

Событие onclick

Код события **onclick** будет выполнен после того, как пользователь щелкнет на элемент, к которому привязано это событие.

```
<script type='text/javascript'>
```

```
function messageShow() {
```

```
    document.write('Вы нажали на кнопку!');
```

```
}
```

```
</script>
```

```
<input type='button' value='Нажмите на кнопку' onclick='messageShow()'/>
```



Событие onload

Код события onload будет выполнен при полной загрузке страницы.

```
<script type='text/javascript'>
```

```
function messageShow() {
```

```
    alert('Страница была загружена!');
```

```
}
```

```
</script>
```

```
<body onload='messageShow()'>
```



События onmouseover и onmouseout

События onmouseover и onmouseout часто используются, чтобы создать "анимированные" кнопки.

Код события onmouseover будет выполнен, когда на элемент, к которому привязано событие, будет наведен курсор мыши.

Код события onmouseout будет выполнен при выведении курсора мыши за пределы элемента.

```
<p id='mes' onmouseover='textChange()' onmouseout='textReturn()'>Наведите на  
этот текст курсор мыши.</p>
```

События мыши

onmousedown — при нажатии кнопки мыши на элементе Web-страницы или самой странице;

□ onmouseup — при отпускании ранее нажатой кнопки мыши;

□ onclick — при щелчке мыши на элементе Web-страницы или на самой Web-странице;

□ ondblclick — при двойном щелчке мыши;

□ onmousemove — при любом перемещении мыши;

□ onmouseover — при наведении курсора мыши на элемент Web-страницы;

□ onmouseout — при выведении курсора мыши с элемента Web-страницы;

□ onselectstart — при начале выделения текста;

□ onselect — при выделении элемента; □ oncontextmenu — при нажатии правой кнопки мыши для вывода контекстного меню.

События клавиатуры

`onkeydown` — при нажатии клавиши на клавиатуре;

- `onkeypress` — аналогично событию `onkeydown`, но возвращает значение кода символа в кодировке Unicode. Наступает постоянно, пока пользователь не отпустит клавишу;
- `onkeyup` — при отпускании ранее нажатой клавиши клавиатуры;
- `onhelp` — при нажатии клавиши `<F1>`.



События документа

onload — после загрузки Web-страницы;

- onscroll — при прокручивании содержимого элемента страницы, документа, окна или фрейма;
- onresize — при изменении размеров окна;
- onbeforeunload — перед выгрузкой документа;
- onunload — непосредственно перед выгрузкой документа. Наступает после события onbeforeunload;
- onbeforeprint — перед распечаткой документа или вывода его на предварительный просмотр;
- onafterprint — после распечатки документа или вывода его на предварительный просмотр.

События формы

`onsubmit` — при отправке данных формы;

- `onreset` — при очистке формы;

- `onblur` — при потере фокуса элементом формы;

- `onchange` — при изменении данных в текстовом поле и перемещении фокуса на другой элемент формы либо при отправке данных формы (наступает перед событием `onblur`);

- `onfocus` — при получении фокуса элементом формы.



Последовательность событий

События возникают последовательно, например, последовательность событий при нажатии кнопкой мыши на элементе страницы будет такой:

```
onmousedown  
onmouseup  
onclick
```

При двойном нажатии последовательность будет такой:

```
onmousedown  
onmouseup  
onclick  
ondblclick
```

Это значит, что событие `ondblclick` возникает после события `onclick`.

При нажатии клавиши на клавиатуре последовательность будет такой:

```
onkeydown  
onkeypress  
onkeyup
```



events и атрибуты событий

DOM объект events содержит дополнительную информацию о событиях, которые произошли.

Атрибуты событий хранятся в DOM объекте events.

С помощью атрибутов событий Вы можете узнать дополнительную информацию о вызове события.

Подробнее о том как можно обратиться к атрибутам событий из скриптов рассказано ниже:

1. После того, как произошло событие создается DOM объект event содержащий атрибуты событий.
2. Объект event должен быть передан функции, которая будет выполняться. Пример: `onclick='funcname(event).'`
3. При объявлении функции необходимо указать, что она принимает объект event. Пример: `function funcname(event){Код функции}`.
4. Теперь Вы можете обращаться к атрибутам событий в коде функции. Пример: `alert(event.shiftKey)`.

events

```
<head>
```

```
<style type='text/css'>
```

```
#block {
```

```
    height: 360px;
```

```
    width: auto;
```

```
    border: 2px dotted #f00;
```

```
}
```

```
p {
```



event

```
text-indent: 40px;
```

```
padding-left: 20px;
```

```
}
```

```
</style>
```

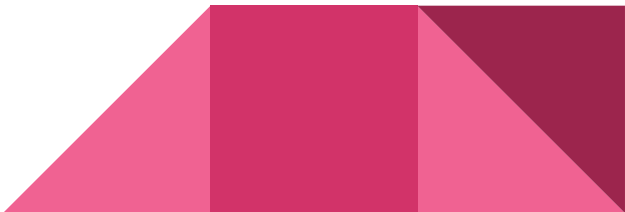
```
<script type='text/javascript'>
```

```
function dispMes(event)
```

```
{
```

```
document.getElementById('par').style.display='block';
```

```
document.getElementById('resultx').innerHTML=event.clientX;
```



event

```
document.getElementById('resulty').innerHTML=event.clientY;
```

```
}
```

```
</script>
```

```
</head>
```

```
<body onclick='dispMes(event)'>
```

```
<div id='block'><p>Щелкните в любом месте правого окна редактора.</p>
```

```
<p id='par' style='display:none;' >Щелчок мыши был произведен по координатам:
```

```
<b><span id='resultx'> </span></b> пикс. по горизонтали от левой границы окна и <br />
```

```
<b><span id='resulty'> </span></b> пикс. по вертикали от верхней границы окна.</p>
```

```
</div>
```

```
</body>
```

Атрибуты событий

`altKey` Позволяет узнать была ли нажата клавиша `Alt` во время вызова события.

`button` Позволяет узнать какая клавиша мыши была нажата во время вызова события. Атрибут имеет значение 0 если была нажата левая кнопка мыши, 1 если была нажата средняя клавиша мыши и 2 если была нажата правая кнопка мыши.

`clientX` Позволяет узнать горизонтальные координаты указателя мыши во время вызова события относительно границ документа.

`clientY` Позволяет узнать вертикальные координаты указателя мыши во время вызова события относительно границ документа.

`ctrlKey` Позволяет узнать были ли нажата клавиша `Ctrl` во время вызова события.

`screenX` Позволяет узнать горизонтальные координаты указателя мыши во время вызова события относительно границ экрана.

`screenY` Позволяет узнать вертикальные координаты указателя мыши во время вызова события относительно границ экрана.

`shiftKey` Позволяет узнать были ли нажата клавиша `Shift` во время вызова события.

`target` Позволяет узнать элемент который вызвал событие.

`type` Позволяет узнать имя события.



Назначение обработчиков событий

в HTML как атрибут тега

```
<input value="Нажми меня" onclick="alert('Клик!)" type="button">
```

в DOM

```
<input id="elem" type="button" value="Нажми меня" />
```

```
<script>
```

```
  elem.onclick = function() {
```

```
    alert( 'Спасибо' );
```

```
  };
```

```
</script>
```



События назначение

назначение через функцию

```
function sayThanks() {  
    alert( 'Спасибо!' );  
}
```

```
elem.onclick = sayThanks;
```

остановка события

```
elem.onclick = null
```



назначение обработчика событий

`<Элемент>.addEventListener(<Событие>, <Ссылка на функцию>, <Перехват>);`

Удалить обработчик события можно с помощью метода `removeEventListener()`. Формат метода:

`<Элемент>.removeEventListener(<Событие>, <Ссылка на функцию>, <Перехват>);`

В параметре `<событие>` указывается название события без префикса "on", например, `click` вместо `onclick`. Ссылка на функцию-обработчик указывается во втором параметре. В эту функцию в качестве аргумента передается ссылка на объект `event`, а внутри функции через ключевое слово `this` доступна ссылка на текущий элемент. Если в параметре `<перехват>` указать значение `true`, то событие будет перехватываться на этапе всплытия от вложенных элементов, а если `false` — то обрабатывается событие самого элемента.

addEventListener

```
<div><span id="span1">span1 <span id="span2">Щелкните здесь (span2)
</span></div> <script type="text/javascript"> function
f_click(e) { // e - ссылка на объект event window.alert("Элемент " +
this.getAttribute("id") + ". Событие возникло в " + e.target.
getAttribute("id")); } if (document.addEventListener) { // В IE не
работает var span1 = document.getElementById("span1"); var span2 =
document.getElementById("span2"); span1.addEventListener("click",
f_click, true); span2.addEventListener("click", f_click, false); }
</script>
```



Всплытие событий

При щелчке на фразе "Щелкните здесь" возникнет последовательность событий:

Элемент `span1`. Событие возникло в `span2`

Элемент `span2`. Событие возникло в `span2`

Таким образом, событие, возникшее во вложенном элементе, вначале обрабатывается элементом-родителем, а затем самим элементом. Если заменить `true` на `false`, то последовательность будет другой:

Элемент `span2`. Событие возникло в `span2`

Элемент `span1`. Событие возникло в `span2`



addEventListener

Позволяет вешать любое количество событий, в отличие от других способов

```
function handler() {  
    alert( 'Спасибо!' );  
}
```

```
input.addEventListener("click", handler);  
// ....  
input.removeEventListener("click", handler);
```



несколько событий

```
<input id="elem" type="button" value="Нажми меня" />
```

```
<script>
```

```
function handler1() {  
    alert('Спасибо!');  
};
```

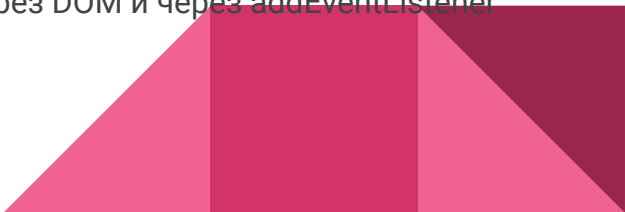
```
function handler2() {  
    alert('Спасибо ещё раз!');  
}
```

```
elem.onclick = function() { alert("Привет"); };
```

```
elem.addEventListener("click", handler1); // Спасибо!
```

```
elem.addEventListener("click", handler2); // Спасибо ещё раз! Назначили и через DOM и через addEventListener
```

```
</script>
```

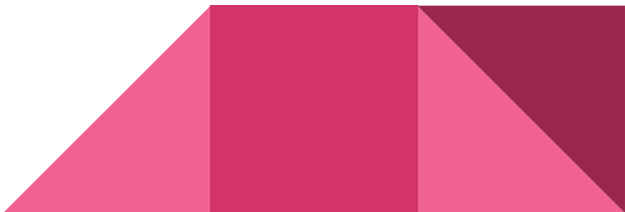


Всплытие

При наступлении события обработчики сначала срабатывают на самом вложенном элементе, затем на его родителе, затем выше и так далее, вверх по цепочке вложенности.

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```



еще раз про event

```
<input type="button" value="Нажми меня" id="elem">
```

```
<script>
```

```
  elem.onclick = function(event) {
```

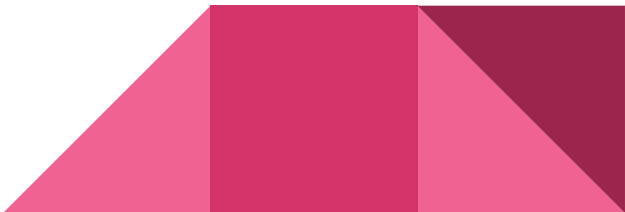
```
    // вывести тип события, элемент и координаты клика
```

```
    alert(event.type + " на " + event.currentTarget);
```

```
    alert(event.clientX + ":" + event.clientY);
```

```
  }
```

```
</script>
```



event

`event.type`

Тип события, в данном случае `click`

`event.currentTarget`

Элемент, на котором сработал обработчик. Значение — в точности такое же, как и у `this`, но бывают ситуации, когда обработчик является методом объекта и его `this` при помощи `bind` привязан к этому объекту, тогда мы можем использовать `event.currentTarget`.

`event.clientX` / `event.clientY`

Координаты курсора в момент клика (относительно окна)



кроссбраузерный вариант event

```
element.onclick = function(event) {  
    event = event || window.event;
```

```
    // Теперь event - объект события во всех браузерах.  
};
```

